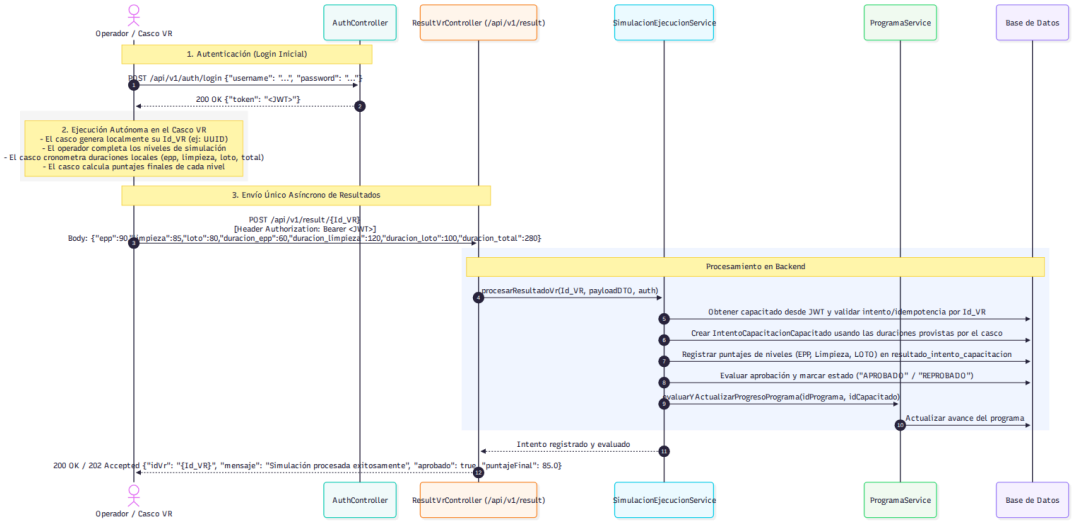
Diagrama de Secuencia - Simulación VR Completamente Asíncrona (Sin Paso de Inicio)

En este modelo, el backend **no requiere un endpoint de pre-inicio (`/iniciar`)**. 1. El usuario se autentica en el casco o dispositivo VR para obtener su Token JWT. 2. El casco genera su propio identificador único de sesión VR (`Id_VR` / UUID local) y ejecuta la simulación por completo de forma autónoma (offline / en dispositivo). El casco cronometra internamente la duración total y la duración de cada nivel. 3. Cuando la simulación termina (o cuando el casco recupera conexión), hace **una única llamada asíncrona** a: `POST /api/v1/result/{Id_VR}` 4. El backend autentica al usuario mediante el JWT provisto en el Header, identifica su asignación activa, crea y cierra el intento en una sola operación atómica o diferida con los tiempos y scores enviados por el casco.

---

1. Diagrama de Secuencia



---

2. Detalle de Peticiones

A. Autenticación (Login)

- **Endpoint:** `POST /api/v1/auth/login`
- **Headers:** `Content-Type: application/json`
- **Body:**

```
{
  "username": "operador_mina",
  "password": "Password123!"
}
```

- **Respuesta Exitosa (200 OK):**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
```

```
  "roles": ["ROLE_CAPACITADO"]
}
```

B. Envío Único de Resultados del Casco VR

El casco genera localmente el `{Id_VR}` (por ejemplo un UUID generado por el visor: `e5b2c12a-8924-4f2a-b7ec-98b671e21b8f`) y lo envía en el path:

- **Endpoint:** `POST /api/v1/result/{Id_VR}`
- **Headers:**
  - `Authorization: Bearer <JWT>`
  - `Content-Type: application/json`
- **Body Request:**

```
{
  "epp": 90.0,
  "limpieza": 85.0,
  "loto": 80.0,
  "duracion_epp": 60,
  "duracion_limpieza": 120,
  "duracion_loto": 100,
  "duracion_total": 280
}
```

- **Respuesta Exitosa (200 OK o 202 Accepted):**

```
{
  "idVr": "e5b2c12a-8924-4f2a-b7ec-98b671e21b8f",
  "mensaje": "Resultado registrado exitosamente",
  "aprobado": true,
  "puntajeFinal": 85.0,
  "duracionTotal": 280
}
```

---

3. Clases Modificadas / Involucradas en el Backend

1. **DTO:**
   - `SimulacionVrResultadoDTO.java`: Contiene los campos del JSON (`epp`, `limpieza`, `loto`, `duracion_epp`, `duracion_limpieza`, `duracion_loto`, `duracion_total`).
2. **Controlador:**
   - `SimulacionEjecucionController.java` (o `SimulacionVrController.java`): Expone `@PostMapping("/api/v1/result/{idVr}")`.
3. **Servicio:**
   - `SimulacionEjecucionService.java`:
     - Resuelve el capacitado desde `Authentication`.
     - Valida si el `{Id_VR}` ya fue procesado para evitar duplicados (idempotencia).
     - Toma los tiempos contados por el casco (`duracion_total`, `duracion_epp`, etc.) calculando `fechaInicio = fechaFin - duracion_total`.
     - Guarda el intento directamente con estado `APROBADO` o `REPROBADO`.
4. **Entidad:**

- `IntentoCapacitacionCapacitado.java`: Campo `codigoSesionVr` para almacenar el `{Id_VR}` generado por el casco.